



Маршрутизация.

i Система спавнера даже на бумаге кажется очень сложной. Простой спавнер, который будет брать врагов из Pool и включать их на точке А, передавать им все точки маршрута и активировать движение - сделать не сложно. Ещё создавая мобов, я знал к чему идти - на сцене будет массив Transform квадратов, которые будут передаваться во врагов через Spawner. Враги будут брать случайную точку поочередно из диапазона квадрата и идти по ним, как бы слегка отклоняясь от идеально ровного курса.

Учитывая, что уровни будут выбираться случайно, а путей может быть больше одного - нужна универсальная система, которая позволит содержать массивы таких путей, а также будет выдавать их в случайном порядке (разные волны смогут идти по разным маршрутам). Добавим сюда же возможность делить волну мобов и пускать её сразу по двум путям в случайном порядке.

Однако, может быть и такое, что если я захочу сделать карту, где два пути не будут пересекаться, а спавнер решит разделить волну попалам - пустит одну часть волны по пути А, другую по пути Б. И быть это может в самом старте игры, при наличии лишь пары башен, покрывающих только один путь. В таком случае, нужно иметь не только обширный арсенал путей и возможность пускать врагов по всем из них, но и возможность ручного управления в некоторых ситуациях. Для этого понадобится двумерный массив, где у каждой волны будет список путей.

+ Если путь не указан → выбирается случайный из общего списка.
Если путь указан → спавнер пустит именно по этому маршруту.
Если путей больше 1+ → можем послать по всем сразу одну волну.

i На поздних этапах (4-5 волны) можно запускать по всем путям сразу, нужно лишь указать их в инспекторе. Если дизайн потребует, чтобы первые волны прошли именно на одном пути → такая возможность будет. Например: будет вторичный путь, на который придётся отвлечься лишь попозже, когда игрок успеет занять башни в той части карты (например на 2-3 волне). Возможно, старт в моей игре окажется слишком быстрым и тогда башен будет в любом случае хватать, а игра покажется очень лёгкой. В финале, когда всё будет работать, меня ждёт ребаланс игры.

- Простой скрипт хранения путей, который будет лежать на каждой сцене. При загрузке сцены спавнер будет находить его сам и брать данные. Хотя можно было прокидывать данные немного иным путём, но этого достаточно.

```

1 using UnityEngine;
2
3 public class SceneEnabler : MonoBehaviour
4 {
5     [Header("Paths for enemies")]
6     public Transform[][] enemyPaths;
7
8     [Header("Custom wave settings")]
9     public int[][] waveCustomPaths;
10
11     [Header("Other settings")]
12     public bool useCustomWaveSettings = false;
13
14 }

```

Сразу же сталкиваемся с проблемой, и узнаём, что Unity не умеет в сериализацию сложных данных. В нашем случае - зубчатых массивов. Он не показывает их в инспекторе. Поэтому придётся сделать обёртки:

```

1 using UnityEngine;
2
3 [System.Serializable]
4 public class EnemyPath
5 {
6     public Transform[] points;
7 }
8
9 [System.Serializable]
10 public class WavePathConfig
11 {
12     public int[] pathIndexes;
13 }
14
15 //ОБНОВЛЕННЫЙ СЦЕН-ЕНЕЙБЛЕР
16 using UnityEngine;
17
18 public class SceneEnabler : MonoBehaviour
19 {
20     [Header("Paths for enemies")]
21     public EnemyPath[] enemyPaths;
22     [Header("Custom wave settings")]
23     public WavePathConfig[] waveCustomPaths;
24     [Header("Other settings")]
25     public bool useCustomWaveSettings = false;
26
27 }

```

```

1 private IEnumerator SpawnWave()
2 {
3     List<EnemyType> waveEnemies = BuildWaveEnemyList();
4

```

```

5     foreach (var enemyType in waveEnemies)
6     {
7         SpawnEnemy(enemyType, currentWavePathIndexes);
8
9         yield return new WaitForSeconds(
10             spawnDelay / SpeedGameManager.Instance.SpeedMultiplier
11         );
12     }
13 }

```

При написании SpawnerManager я столкнулся с осознанием следующей проблемы. Я использовал встроенный WaitForSeconds, куда передавал значения задержек. Однако некоторые задержки должны также учитывать текущую игровую скорость - SpeedMultiplier. Если условный таймер пред-волны начинает отчёт от 7 секунд, то на скорости 0 он должен застыть. Изначально я не учёл этого, и на паузе UI текст таймера менялся, как и Wait.

Следующие проблемы: при делении задержки на текущую скорость игры, мы, конечно, сократим время спавна (ускорим игру). Однако если щелкать множители времени на ходу - Wait не обновится, потому что данные мы уже передали в метод и он уже выполняется.

Решение простое. Мне не понадобится вручную перевычислять какие-то данные. Я сделал свой кастомный Wait аналог:

```

1 private IEnumerator WaitScaled(float duration)
2 {
3     float timer = duration;
4
5     while (timer > 0f)
6     {
7         timer -= Time.deltaTime *
8             SpeedGameManager.Instance.SpeedMultiplier;
9         yield return null;
10    }
11 }
12 /// Меняем старую запись на нашу функцию.
13 yield return new WaitForSeconds(...); =>
14     yield return WaitScaled(...);

```

Я могу работать с таким таймером отчёта как с обычным Wait, останавливая его даже во время выполнения, потому что это IEnumerator. Теперь таймер отнимает не просто deltaTime, а ещё и умножает прошедшее время на текущую скорость игры, и если скорость будет 0 (пауза) - то и таймер перестанет отниматься, а короутина просто будет крутиться в ожидании, когда игровое время снова пойдёт.

Если бы этот кастомный таймер понадобился бы в других местах игры → стоило бы его переместить в отдельный статик класс, чтобы использовать и в других местах.

EnemyRole.



Для контроля спавна волн, я создал простое деление на роли, чтобы контролировать спавн чуть более детально:

```
1 enum EnemyRole
2 {
3     Boss,
4
5     Fast_1,
6     Fast_2,
7     Fast_3,
8
9     Tank_1,
10    Tank_2,
11    Tank_3,
12
13    Swarm_1,
14    Swarm_2,
15    Swarm_3
16 }
```

Каждому мобу вручную присваивается EnemyRole, в зависимости от его характеристик. Позже это будет использоваться для генерации волн.

SpawnerManager.

```
1 using System.Collections;
2 using System.Collections.Generic;
3 using TMPro;
4 using UnityEngine;
5
6 public class EnemySpawnerManager : Manager<EnemySpawnerManager>
7 {
8     [Header("Demo Spawn Settings")]
9     [SerializeField] private EnemyType[] enemiesToSpawn;
10    [SerializeField] private int countPerEnemy = 2;
11    [SerializeField] private float spawnDelay = 1.33f;
12
13    [Header("Wave Timings")]
14    [SerializeField] private float preWaveDelay = 10f;
15    [SerializeField] private float postWaveDelay = 15f;
16
17    [Header("UI")]
18    [SerializeField] private GameObject timerWaveCanvas;
19    [SerializeField] private TextMeshProUGUI timerTXT;
20
21    [Header("Path Preview")]
22    [SerializeField] private GameObject skullPreviewPrefab;
23    [SerializeField] private float previewRepeatDelay = 0.7f;
24    [SerializeField] private float previewSpawnDelay = 0.02f;
25
26    private int[] aliveEnemiesPerWave;
27    private bool[] waveSpawnFinished;
28    private bool[] waveCleared;
29
30    public int CurrentWave { get; private set; } = 0;
31    private int maxWave = 5;
32
33    private SceneEnabler currentSceneData;
34    private bool isSpawning = false;
35    private Coroutine waveLoopCoroutine;
36
37    private int[] currentWavePathIndexes;
```

```

38
39     private float[] alphasPreview = { 1f, 0.65f, 0.45f };
40
41     // -----
42     // INITIALIZATION
43     // -----
44
45     /*
46      SceneEnabled(int) - вызывается из GameFlowManager при загрузке сцены
47      По стандарту 5 волн, но можно прокинуть любое значение. Ищем у текущей
48      сцены SceneEnabler и берём его данные о путях. Вместе с тем, сразу же
49      инициализируем WaveGenerator, чтобы тот позаботился и заранее просчита
50      врагов.
51
52      StartSpawning вызывается уже при активации состояния Playing
53      (нажатии кнопки на пре-старт экране). Он запускает корутину WaveLoop.
54      */
55     public void SceneEnabled(int MaxWave = 5)
56     {
57         currentSceneData = FindObjectOfType<SceneEnabler>();
58
59         if (currentSceneData == null) { Debug.LogError("SceneEnabler n
60
61         maxWave = MaxWave;
62
63         HudManager.Instance.SetTotalWaveIcons(maxWave);
64
65         WaveGenerator.Instance.InitializationGenerator(maxWave, Diffic
66
67         ResetNotify(MaxWave);
68     }
69
70     // -----
71     // PUBLIC API
72     // -----
73
74     public void StartSpawning()
75     {
76         if (isSpawning) return;
77
78         if (currentSceneData == null) { Debug.LogError("No SceneEnable
79             return;
80         }
81         isSpawning = true;
82         waveLoopCoroutine = StartCoroutine(WaveLoop());
83     }
84
85     public void StopSpawning()
86     {
87         if (!isSpawning && waveLoopCoroutine == null)
88             return;
89
90         isSpawning = false;
91         if(waveLoopCoroutine != null)
92         {
93             StopCoroutine(waveLoopCoroutine);
94             waveLoopCoroutine = null;
95         }
96
97         CurrentWave = 0;
98         HudManager.Instance.TimerWaveHide();
99     }
100
101
102     // -----
103     // SPAWN WAVE
104     // -----

```

```

105
106     private IEnumerator SpawnWave()
107     {
108         List<EnemyType> waveEnemies =
109             WaveGenerator.Instance.GetEnemyWaveList(CurrentWave-1);
110
111         if (waveEnemies == null || waveEnemies.Count == 0)
112         {
113             Debug.LogWarning("Wave is empty!");
114             yield break;
115         }
116
117         foreach (var enemyType in waveEnemies)
118         {
119             SpawnEnemy(enemyType, currentWavePathIndexes);
120
121             yield return WaitScaled(spawnDelay);
122         }
123         waveSpawnFinished[CurrentWave] = true;
124     }
125
126     // Далее следует SpawnWave цикл. Собственно, сам спавн врагов.
127     // Мы получаем из генератора по текущему индексу волны List мобов. Пер
128     // каждого моба в метод SpawnEnemy, делаем небольшую задержку и продол
129
130     // -----
131     // POST WAVE
132     // -----
133
134     private IEnumerator PostWaveDelay()
135     {
136         yield return WaitScaled(postWaveDelay);
137     }
138
139     // PostWave - последняя часть цикла, в виде перерыва перед спавном сле
140
141     // -----
142     // PATH SELECTION
143     // -----
144
145     private int[] GetPathsForWave()
146     {
147         //Customs
148         if (currentSceneData.useCustomWaveSettings &&
149             currentSceneData.waveCustomPaths != null &&
150             CurrentWave - 1 < currentSceneData.waveCustomPaths.Length)
151         {
152             var custom = currentSceneData.waveCustomPaths[CurrentWave
153
154             // Random
155             if (custom == null || custom.Length == 0)
156             {
157                 return GetRandomSinglePath();
158             }
159
160             return custom;
161         }
162
163         return GetRandomSinglePath();
164     }
165
166     private int[] GetRandomSinglePath()
167     {
168         int index = Random.Range(0, currentSceneData.enemyPaths.Length
169         return new int[] { index };
170     }
171

```

```

172  /*
173   GetPathsForWave - выдаёт маршрут для текущей волны. Если в данных Sc
174 мы видим для текущей волны установку флага custom, значит для данной в
175 отмечен вручную. Мы просто возвращаем его. Если же путь вручную не ука
176 значит будем получать случайный из доступных. Если стоит галочка custo
177 указано два или более пути, значит для данной волны предполагается слу
178 распределение врагов по всем выделенным маршрутам.
179 GetRandomSinglePath - берёт случайный путь из доступных.
180 */
181
182 // -----
183 // ENEMY SPAWN
184 // -----
185
186 /*
187 Как происходит спавн врагов? SpawnEnemy получает тип врага (вариант)
188 путей. Мы получаем из PoolManager нужный тип врага, затем получаем слу
189 из пришедших. В большинстве случаев это массив на один элемент. Если ж
190 указано больше, то каждый враг может быть случайно заспавнен на одном
191 маршрутов. В массив Transform присваиваем по индексу нужные объекты с
192 диапазонами под точки.
193
194 Перемещаем объект в стартовую позицию. Получаем его компоненты.
195 Передаём врагу текущую волну (это нужно для учёта уничтоженных врагов
196 волны). Увеличиваем массив живых врагов по индексу волны. Передаём пут
197 делаем врага активным и запускаем движение.
198 */
199 private void SpawnEnemy(EnemyType type, int[] pathIndexes)
200 {
201     var go = PoolManager.Instance.Get(type.ToString());
202
203     if (go == null) return;
204
205     int pathIndex = pathIndexes[Random.Range(0, pathIndexes.Length
206
207     Transform[] path =
208         currentSceneData.enemyPaths[pathIndex].points;
209
210     if (path == null || path.Length == 0)
211     {
212         Debug.LogError($"Path {pathIndex} is empty");
213         return;
214     }
215
216     go.transform.position = path[0].position;
217
218     var movement = go.GetComponent<EnemyMovement>();
219     var controller = go.GetComponent<EnemyController>();
220
221     if (controller == null)
222     {
223         Debug.LogError("EnemyController is Null! Spawn canceling!");
224         return;
225     }
226     controller.SetWaveIndex(CurrentWave);
227     aliveEnemiesPerWave[CurrentWave]++;
228
229     movement.SetPath(path);
230     go.SetActive(true);
231     movement.StartMovement();
232 }
233

```



На момент написания документации я понял, что следует улучшить систему. Сейчас по двум путям можно пойти только при указании их вручную и установке

флага Custom. Думаю, следует добавить флаг RandomMultiplePath[] - при true для индекса волны будет случайно выбираться - по одному пути пойдут mobs или несколькими. RandomMultiplePath[3] = true; Означает, что с 4 волны есть шанс, что mobs пойдут по нескольким путям сразу. В связи с тем, что большие карты не планируются, вряд ли будет более трёх путей на карте и ограничивать, или добавлять RandomDoublePath не имеет смысла для данного проекта.

Когда враг умирает, он вызывает метод NotifyEnemyKilled и передаём индекс волны, к которой он принадлежит. В случае, если все враги волны уничтожены, то в GameManager мы записываем +1 волну. В конце забега нам понадобится это, чтобы выводить данные об уничтоженных волнах.

К такому решению я пришёл не сразу, оно создаёт жесткую связку, но зато решение простое в реализации. Я рассматривал иные варианты, чтобы не писать сложный код, и чтобы всё работало также просто, но многие упирались в проблемы. Если хранить только мобов, то мы никогда не запишем + волну, потому что враги могут начать спавниться из второй волны до уничтожения первой. А я не хотел менять Game Flow логику. Я хочу чтобы mobs могли спавниться даже до уничтожения предыдущей волны.

Можно было бы просто при начале каждой волны делать +волну. В таком случае, даже если бы волна не была уничтожена, она всё равно бы попадала в статистику. Да, погрешность от реальных данных = 1, редко 2, волнам. Хотелось всё-таки сделать рабочую систему. Получилось такое решение.

```
1 // -----
2 // ENEMY DIE
3 // -----
4 public void NotifyEnemyKilled(int waveIndex)
5 {
6     aliveEnemiesPerWave[waveIndex]--;
7     if (waveSpawnFinished[waveIndex] &&
8         !waveCleared[waveIndex] &&
9         aliveEnemiesPerWave[waveIndex] == 0)
10    {
11        waveCleared[waveIndex] = true;
12        GameManager.Instance.WaveCleared();
13    }
14 }
15
16 private void ResetNotify(int maxWave = 5)
17 {
18     aliveEnemiesPerWave = new int[maxWave+1];
19     waveSpawnFinished = new bool[maxWave+1];
20     waveCleared = new bool[maxWave+1];
21 }
22
23 // -----
24 // CUSTOM WAIT
25 // -----
26
27 private IEnumerator WaitScaled(float duration)
```

```

28     {
29         float timer = duration;
30
31         while (timer > 0f)
32         {
33             timer -= Time.deltaTime *
34                 SpeedGameManager.Instance.SpeedMultiplier;
35             yield return null;
36         }
37     }
38 }

```

WaveGenerator.

i Менеджер спавна решает только когда нужно спавнить. Кого спавнить будет решать WaveGenerator.

Лист врагов сейчас указывается вручную. В ArtefactsManager я использовал более продвинутое продакшен-стайл решение: Addressables. Позже возможно я заменю и эту часть кода, но для документации пока оставлю так.

```

1  using System.Collections.Generic;
2  using UnityEngine;
3
4  public class WaveGenerator : Manager<WaveGenerator>
5  {
6      [Header("EnemyPrefabs")]
7      [SerializeField] private List<GameObject> enemyPrefabs;
8
9      private Dictionary<EnemyRoleTier, List<EnemyType>> roleMap;
10     private Dictionary<EnemyType, EnemyStats> statsMap;
11
12     // Создаём два словаря.
13     // Первый будет хранить роль и мобов, что относятся к данной роли.
14     // Например: tank_1: Plant1, Goblin1, Rot.
15     // Второй будет хранить тип каждого моба и его EnemyStats (для вычисления
16
17     private List<List<EnemyType>> waves;
18     private List<int> waveBudgets;
19
20     private int maxWaveCount = 5;
21
22     private void Start()
23     {
24         InitMap();
25     }
26
27     private void InitMap()
28     {
29         statsMap = new();
30         roleMap = new();
31
32         foreach (var prefab in enemyPrefabs)
33         {
34             var stats = prefab.GetComponent<EnemyStats>();
35
36             statsMap[stats.type] = stats;
37
38             if (!roleMap.ContainsKey(stats.role))
39             {
40                 roleMap[stats.role] = new List<EnemyType>();

```

```

41     }
42     roleMap[stats.role].Add(stats.type);
43 }
44 }
45
46 // InitMap - инициализируем карты. С каждого префаба получаем его Enemy
47 // EnemyType, EnemyRoleTier и связываем всё это дело в две удобные карты
48
49 public void InitializationGenerator(int maxWave, int weightFloor)
50 {
51     maxWaveCount = maxWave;
52
53     waves = new List<List<EnemyType>>(maxWave);
54
55     waveBudgets = BuildWaveBudgets(maxWave, weightFloor);
56
57     for (int wave = 0; wave < maxWave; wave++)
58     {
59         waves.Add(BuildWave(wave));
60     }
61 }
62
63 // InitializationGenerator(int) - инициализируем генератор,
64 // передавая в него максимальное число волн, а также вес волн для теку
65 // (берётся из DifficultyManager). Для каждой волны вызываем BuildWave
66 // В итоге у нас должен будет получиться массив из нескольких листов вра
67
68 private List<int> BuildWaveBudgets(int maxWave, int totalWeight)
69 {
70     List<int> result = new();
71
72     int sum = 0;
73     for(int i = 1; i <= maxWave; i++)
74     {
75         sum += i;
76     }
77
78     int allocated = 0;
79
80     for (int i = 1; i <= maxWave; i++)
81     {
82         int budget = totalWeight * i / sum; //3000: 200, 400, 600,
83         result.Add(budget);
84         allocated += budget;
85     }
86
87     result[result.Count - 1] += totalWeight - allocated;
88     return result;
89 }
90
91 /*
92  BuildWaveBudgets - возвращает лист веса под каждую волну.
93  Пока что используется вычисление ниже: мы берём общий вес, умножаем на
94  волну и делим на sum (треугольное число, для 5 = 1+2+3+4+5 = 15). В ср
95  градация веса волны получается приемлемой, но в будущем возможны правк
96  */
97
98
99 /*
100  BuildWave - получает индекс волны. Веса мы уже записали в массив и
101  работать с ними по индексам волн. Генерируем модификаторы через Genes
102  Далее делаем проверку на IsBossWave, а также проверяем есть ли вообще в
103  роли босса какие-нибудь префабы. Если это волна последняя, то она в лю
104  получит босса.
105
106  Начинаем сборку: будем собирать волну, пока не кончиться бюджет. Пол
107  если роль открыта - получаем врага по роли. Если роль закрыта для

```

```

108 текущей волны → пропускаем итерацию. Получаем вес моба из EnemyStats
109 (Ранее это было Exp врага (здоровье * скорость + урон), но в итоге я п
110 абстрактную модель веса, которая вписывается руками и работает не от 0
111
112 Теперь прокидываем роль внутрь GetGroupCount, чтобы получить не прос
113 моба, а пачку (так волна смотрится красивее и читаемее). Вычитаем из в
114 вес каждого и добавляем их в лист.
115 */
116 private List<EnemyType> BuildWave(int waveIndex)
117 {
118     int budget = waveBudgets[waveIndex];
119     List<EnemyType> result = new();
120
121     HashSet<WaveModifier> modifiers =
122         GenerateModifiers(waveIndex);
123
124     if (IsBossWave(waveIndex) && roleMap.ContainsKey(EnemyRoleTier
125     {
126         EnemyType boss = PickBoss();
127         int bossCost = statsMap[boss].cost;
128
129         if (bossCost <= budget)
130         {
131             result.Add(boss);
132             budget -= bossCost;
133         }
134     }
135
136     int safetyCounter = 3000;
137
138     while (budget > 0 &&
139         safetyCounter-- > 0 &&
140         HasEnemyFitBudget(budget))
141     {
142         EnemyRoleTier role = PickRoleByWave(waveIndex);
143
144         // роль открыта.
145         if (!IsRoleUnlocked(role, waveIndex))
146             continue;
147
148         // для роли есть враги.
149         if (!roleMap.ContainsKey(role) || roleMap[role].Count == 0
150             continue;
151
152         EnemyType type = PickEnemy(role);
153         int cost = statsMap[type].cost;
154
155         // Если не помещается в бюджет
156         if (cost > budget)
157             continue;
158
159         int count = GetGroupCount(
160             role,
161             waveIndex,
162             modifiers);
163
164         for (int i=0; i<count; i++)
165         {
166             if (cost > budget)
167             {
168                 break;
169             }
170             result.Add(type);
171             budget -= cost;
172         }
173     }
174

```

```

175         return result;
176     }
177
178     //////////////////////////////////////
179     /// Role Selection
180     //////////////////////////////////////
181     private EnemyRoleTier PickRoleByWave(int wave)
182     {
183         int roll = Random.Range(0, 100);
184
185         if (wave == 0)
186             return EnemyRoleTier.Swarm_1;
187
188         if (wave <= 1)
189         {
190             if (roll < 70) return EnemyRoleTier.Swarm_1;
191             if (roll < 90) return EnemyRoleTier.Tank_1;
192             return EnemyRoleTier.Fast_1;
193         }
194
195         if (wave <= 2)
196         {
197             if (roll < 50) return EnemyRoleTier.Swarm_2;
198             if (roll < 75) return EnemyRoleTier.Tank_2;
199             return EnemyRoleTier.Fast_2;
200         }
201
202         return (EnemyRoleTier)Random.Range(0, roleMap.Count);
203     }
204
205     // PickRoleByWave - сортирует роли по этажам. К примеру, на первом эта
206     // только враги с ролью Swarm_1 (рой). После вычисления роли берём из
207     // случайного юнита подходящей роли.
208
209     //////////////////////////////////////
210     /// Helpers
211     //////////////////////////////////////
212     private bool IsBossWave(int wave)
213     {
214         return wave == maxWaveCount - 1;
215     }
216
217     private EnemyType PickEnemy(EnemyRoleTier role)
218     {
219         var pool = roleMap[role];
220         return pool[Random.Range(0, pool.Count)];
221     }
222
223     // PickEnemy - возвращаем EnemyType нужного моба.
224
225     private EnemyType PickBoss()
226     {
227         var bosses = roleMap[EnemyRoleTier.Boss];
228         return bosses[Random.Range(0, bosses.Count)];
229     }
230
231     private bool IsRoleUnlocked(EnemyRoleTier role, int wave)
232     {
233         return role switch
234         {
235             EnemyRoleTier.Swarm_1 => true,
236             EnemyRoleTier.Tank_1 => wave >= 0,
237             EnemyRoleTier.Fast_1 => wave >= 0,
238
239             EnemyRoleTier.Tank_2 => wave >= 1,
240             EnemyRoleTier.Swarm_2 => wave >= 1,
241

```

```

242
243         EnemyRoleTier.Tank_3 => wave >= 2,
244         EnemyRoleTier.Fast_2 => wave >= 2,
245         EnemyRoleTier.Swarm_3 => wave >= 2,
246
247         EnemyRoleTier.Fast_3 => wave >= 3,
248
249         EnemyRoleTier.Boss => wave >= 4,
250
251         _ => false
252     };
253 }
254
255 // IsRoleUnlocked(EnemyRoleTier, int) - проверяет доступна ли роль для
256 // Это ограничение ролей, чтобы в первых волнах не могли спавниться са
257 // и быстрые враги.
258
259 private bool HasEnemyFitBudget(int budget)
260 {
261     foreach (var stats in statsMap.Values)
262     {
263         if (stats.cost <= budget)
264         {
265             return true;
266         }
267     }
268     return false;
269 }
270 // HasEnemyFitBudget - проверяет, существует ли хотя бы ещё один моб,
271 // можно купить на оставшийся бюджет. Если да - возвращает true и мы п
272 // цикл заполнения волны.
273
274 ////////////////////////////////////
275 /// Grouping
276 ////////////////////////////////////
277
278 /*
279 GetGroupCount(EnemyRoleTier, int, HashSet<WaveModifier>) - главный с
280 Сначала он обрабатывает модификаторы, которые могут выпасть для волны,
281 проверяет на выпавшую роль и выдаёт случайное количество мобов в завис
282 волны. Например: на 5 волне выпавшая роль танка второго уровня подразу
283 спавн толпы из 3-6 мобов. Таким образом собираются небольшие толпы, вс
284 сумируются и получается красивая волна.
285
286 Этот метод придётся расширять, по мере появления нового функционала
287 способов группировочной сборки. Условно, если захочется добавить хаот
288 модификаторы, или логику под большее количество волн - код разрастётся
289 */
290
291 private int GetGroupCount(EnemyRoleTier role, int wave, HashSet<Wa
292 {
293     if (modifiers.Contains(WaveModifier.SwarmRush))
294     {
295         if (role == EnemyRoleTier.Swarm_1)
296         {
297             return Random.Range(8, 14);
298         }
299         if (role == EnemyRoleTier.Swarm_2)
300         {
301             return Random.Range(7, 12);
302         }
303         if (role == EnemyRoleTier.Swarm_3)
304         {
305             return Random.Range(5, 10);
306         }
307     }
308 }

```

```

309     if (modifiers.Contains(WaveModifier.TankOverflow))
310     {
311         if (role == EnemyRoleTier.Tank_1)
312             return Random.Range(6, 12);
313
314         if (role == EnemyRoleTier.Tank_2)
315             return Random.Range(5, 10);
316     }
317
318     if (modifiers.Contains(WaveModifier.FastRush))
319     {
320         if (role == EnemyRoleTier.Fast_1)
321             return Random.Range(6, 8);
322
323         if (role == EnemyRoleTier.Fast_2)
324             return Random.Range(4, 7);
325     }
326
327     switch (role)
328     {
329         case EnemyRoleTier.Swarm_1:
330             return Random.Range(4, 8);
331         case EnemyRoleTier.Swarm_2:
332             return Random.Range(3, 7);
333         case EnemyRoleTier.Swarm_3:
334             return Random.Range(3, 6);
335
336
337         case EnemyRoleTier.Tank_1:
338             if (wave <= 1)
339             {
340                 return Random.Range(1, 3);
341             }
342             if (wave <= 3)
343             {
344                 return Random.Range(3, 6);
345             }
346             return Random.Range(2, 4);
347
348         case EnemyRoleTier.Tank_2:
349             if (wave <= 2)
350             {
351                 return Random.Range(2, 3);
352             }
353             if (wave <= 4)
354             {
355                 return Random.Range(3, 6);
356             }
357             return Random.Range(2, 3);
358
359         case EnemyRoleTier.Tank_3:
360             return 1;
361
362         case EnemyRoleTier.Fast_1:
363             if (wave <= 2)
364             {
365                 return Random.Range(2, 4);
366             }
367             if (wave <= 4)
368             {
369                 return Random.Range(4, 8);
370             }
371             return Random.Range(2, 4);
372
373         case EnemyRoleTier.Fast_2:
374             if (wave <= 2)
375             {

```

```

376         return Random.Range(1, 3);
377     }
378     if (wave <= 4)
379     {
380         return Random.Range(3, 5);
381     }
382     return Random.Range(1, 2);
383
384     case EnemyRoleTier.Fast_3:
385         return 1;
386
387     }
388     return 1;
389 }
390
391 ///////////////////////////////////////////////////
392 /// GETING
393 ///////////////////////////////////////////////////
394
395 // К началу игры генератор уже сгенерировал массив листов с порядком с
396 // Если ничего не случилось, никаких ошибок нет, то мы вернём по запросу
397 // List мобов по индексу волны.
398
399
400
401 public List<EnemyType> GetEnemyWaveList(int currentWave)
402 {
403     if (waves == null || currentWave < 0 || currentWave >= waves.Count)
404     {
405         Debug.LogError("WavesList is Null!");
406         return new List<EnemyType>();
407     }
408     return waves[currentWave];
409 }
410
411 // GenerateModifiers(int) - который случайно может выкинуть
412 // модификатор, и в таком случае волна получит определенный паттерн.
413 // Например: волна танков.
414
415 private HashSet<WaveModifier> GenerateModifiers(int wave)
416 {
417     HashSet<WaveModifier> modifiers = new();
418
419     if (Random.value < 0.08f)
420     {
421         modifiers.Add(WaveModifier.TankOverflow);
422     }
423
424     if (Random.value < 0.16f)
425     {
426         modifiers.Add(WaveModifier.SwarmRush);
427     }
428
429     if (Random.value < 0.06f)
430     {
431         modifiers.Add(WaveModifier.FastRush);
432     }
433
434     return modifiers;
435 }
436
437 }
438
439
440 public enum WaveModifier
441 {
442     TankOverflow,

```

```
443     SwarmRush,  
444     FastRush  
445 }
```

- ❖ Функционал спавнера можно долго расширять. Бесконечность предполагает составление случайных волн, однако необходимо и наличие паттернов. Как например - спавн танков в первой волне в размере 1-3. Такие паттерны можно добавлять долго и увеличивать разнообразие генерации.
- ✓ На данном этапе волна получается приемлемой и играбельной. Выглядит красиво, по виду чувствуется сбалансированной. В процессе финального баланса что-то может измениться, но результатом я доволен.